

STATE//LESS AI 활용 기술 문서

1. 개요

STATE//LESS — SIGNAL STRIKE는 6개 슬롯에 무작위로 켜지는 신호 중 가짜만 재빨리 정확하고 진짜는 흘려보내는 60초 리플렉스 아케이드입니다. 개발 보조 도구로 **OpenAI Codex**와 **Claude Code(Anthropic)**를 사용했습니다 — 핵심 메커닉이 여러 차례 바뀌는 동안 Codex와 진행하다가, 사전과제 마감을 앞두고 Claude Code로 전환해 현재 SIGNAL STRIKE 구조까지 이어왔습니다. 두 도구 모두 게임 콘셉트 구체화, 프론트엔드 구현, 로직 테스트, 접근성 점검, 문서 작성에 활용했습니다.

런타임 게임 자체는 외부 생성형 AI API를 호출하지 않습니다. 신호의 종류·등장 간격·지속시간·강화 효과는 브라우저 안의 결정적 규칙 엔진이 경과 시간과 저장된 진행도만으로 계산합니다 — 심사 환경에서 API 키, 비용, 응답 지연, 네트워크 장애에 의존하지 않도록 하기 위한 설계입니다.

이 문서에서 AI는 “무엇을 만들게 시켰는가”보다 **무엇을 검증하고 어떻게 판단을 바꿨는가**로 활용했습니다. 개발 전 기간 같은 패턴이 반복됩니다: 결과물을 의심 → 이론 조사·실제 플레이·소스 대조로 근거를 만들거나 검증 → 그 근거로 설계를 바꿈. 이 판단 과정 자체는 최종 게임에 남아 있지 않은 여러 메커닉(\$4)을 거쳤지만, 그 과정에서 확립된 검증 습관(\$3)은 현재 게임에도 그대로 이어지고 있습니다.

2. 최종 게임에 반영된 AI 활용

아래는 지금 배포된 빌드(<https://softkleenex.github.io/state-less-game/>) 기준으로, 기능 영역별 요청 → 반영 → 검증을 정리한 것입니다.

2.1 핵심 메커닉 — SIGNAL STRIKE 반응형 판정

게임처럼 느껴지지 않는 건 문답 부분이고, 기록(텍스트)을 대조하는 게 게임이라기보다는 문서작업에 가까워 보여. 아케이드 게임인데, 메타적 요소를 가진 아케이드나 미니게임은 어때?

이전까지의 판정 방식(텍스트 주장을 읽고 대조)이 “본질적으로 문서 작업”이라는 지적에 따라, 판정 자체를 색·기호 즉시 판별로 교체했습니다. 6개 고정 슬롯에 무작위로 **rose × (가짜)** 또는 **cyan ◆ (진짜)** 신호가 짧게 켜지고, 가짜만 사라지기 전에 클릭·번호 키(1~6)로 정확하며 진짜는 그대로 흘려보내는 60초 단일 런으로 설계했습니다. 난이도는 경과 시간만의 함수인 세 곡선(등장 간격 900ms→340ms, 지속시간 1.45초→0.7초, 동시 개수 1→4)으로 결정되며, 단일 `requestAnimationFrame` 루프(`gameLoop`)가 슬롯 만료·스폰·60초 종료를 모두 처리합니다.

검증: Puppeteer로 실제 크롬을 띄워 가짜 클릭 시 정확·점수·콤보 상승, 진짜 클릭 시 무결성 하락과 콤보 초기화, 오클릭 3회 시 `NULL` 랭크로 결과 화면 도달, 번호 키 1~6이 클릭과 동일하게 동작하는지 확인 — 콘솔 에러 0건.

2.2 로그라이트 강화 시스템 — 무제한 스택 46종

이게임,,, 재밌냐? 단순히 순발력 말고는 무슨 의미가 있는지 모르겠네. 아이템(업그레이드)를 좀더 다방면으로 만들어버리는건 어때...? 뱀서류 처럼 하는거지. 뱀서류랑 다른 건 아이템 개수에 제한은 아예 없는 걸 특징으로 삼자.

반응형 판정만으로는 매 순간의 결정에 트레이드오프가 없다는 지적에 따라, 60초 런 위에 로그라이트 강화 선택을 엮었습니다. 처음에는 15/30/45초 3회·6종 소진형으로 시작했다가, “뱀서류처럼, 근데 개수 제한 없이”라는 요청에 맞춰 **7초부터 6.5초 간격으로 반복되는 픽(런당 약 8회)**으로 바꾸고, 같은 강화를 몇 번이든 다시 뽑아 무제한으로 스택할 수 있게 했습니다 — 슬롯 제한·교체가 있는 일반적인 로그라이트와의 명시적 차이점입니다.

콘텐츠는 두 경로로 만들었습니다. 새 런타임 동작이 필요한 시그니처 메커닉 6종(보호막·부활·연쇄 정확·마일스톤 고정 보너스·DEEP VERIFY 추가 사용권 2종)은 직접 설계·구현했고, 나머지 30종은 병렬 서버에이전트 3개(공격/방어/와일드카드 테마)에

허용된 effects 필드와 “모든 강화는 업사이드와 다운사이드를 함께 가진다”는 원칙을 프롬프트로 명시해 초안을 받은 뒤 전부 직접 검수(필드 오남용·업사이드 단독·과한 밸런스 확인)했습니다. 기존 10종을 더해 최종 46종입니다.

무제한 스택이 수식을 깨지 않도록, 개수를 제한하는 대신 모든 소비 지점에 하한선을 뒀습니다 — 콤보/점수 배율은 0/0.15 밑으로, 신호 지속시간은 220ms, 스폰 간격은 90ms, DEEP VERIFY 창은 1초 밑으로 내려가지 않습니다. 코어 주위를 도는 `#buff-orbit` 링과 `#resource-strip` 으로 보유 강화·자원을 한눈에 보이게 했고, 46종 전부 MORI 반응 대사(`moriLine`)를 채웠습니다.

검증 중 발견한 버그: Puppeteer로 보호막 픽 전/후 상태를 대조하는 과정에서, `runtime.shieldCharges` 는 정확히 소모되고 있었지만 `#resource-strip` 표시가 픽 시점 이후 갱신되지 않는 버그를 발견 — `updateActiveBufsReadout()` 를 매 HUD 갱신(`updateHud()`)에서도 호출하도록 고쳤습니다. “동작한다”에서 멈추지 않고 전/후 상태를 직접 대조했기 때문에 잡힌 문제입니다.

검증: 60초 런 자동 플레이로 약 8회 픽이 반복 간격대로 열리는지, 같은 강화를 두 번 골랐을 때 `×2` 표기가 정확한지(실측: 8회 픽·55,007점·S랭크), 보호막이 실제로 오클릭을 무효화하는지, DEEP VERIFY 추가 사용권으로 런당 두 번째 발동이 가능한지 확인 — 콘솔 에러 0건.

2.3 난이도 곡선 이징 + 적응형 난이도

본 리플렉스 반응 자체의 전체 난이도 곡선, 초반 단계에서 가짜 신호가 너무 빨라. 신호를 방어에 실패한다면 그 신호에 따라서 조정 가능한 게 조정되는 컨셉은 어때?

두 가지를 함께 반영했습니다. **이징:** 세 escalation 곡선이 경과 비율을 그대로 선형 보간에 넣어 초반부터 체감 난이도가 빠르게 올랐던 것을, 비율을 1.5제곱(`easeEscalationRatio`)해 넣는 것으로 바꿔 시작·종료 난이도는 그대로 두고 초반만 완만하게 했습니다. **적응형 난이도:** “실패에 따라 조정되는 컨셉”을 재확인해, 숨겨진 성과 미터(`performanceMeter`, -6~+6, 정화 성공 +1·실수 -2)가 스폰 간격·신호 지속시간을 최대 $\pm 12\%$ 까지만 엇는 `getAdaptiveDifficultyScale` 을 추가했습니다. 회복(정화 6회 필요)보다 완화(실수 2회로 바닥)가 더 빠른, 봐주는 방향의 비대칭입니다. 60초 고정 곡선을 대체하지 않고 그 위에 살짝 얹히기만 합니다.

검증: 이징된 곡선이 10초 지점에서도 최소 동시 개수(1개)를 유지하는지, 적응형 스케일이 ± 6 구간에서 대칭적으로 상하한에 도달하는지 순수 함수 테스트로 확인.

2.4 MORI 캐릭터 바이블·대사·튜토리얼

mori의 대사를 좀더 다양하게 하고, 대사가 밖에 뜨는 걸 모르는 사람들도 있을 것 같아. 간단한 튜토리얼도 넣으면 좋겠는데. UI, UX 및 캐릭터성도 좀더 살펴보자, 예를들면 튜토리얼에서 모리가 말하는 것처럼.

MORI의 캐릭터성을 1인칭 반말, 짧고 건조한(`deadpan`) 어조, 감탄사·존댓말 금지, 절제된 응원으로 명문화하고, 기존 코드(46종 강화 대사 포함)를 이 기준으로 전수 검사해 일관성을 확인했습니다. 고빈도 상태(관찰·정답·오답 등)마다 대사 변형을 4~5개씩 추가해 매번 다른 문장이 나오게 했고, 모든 대사가 한 글자씩 타이핑되며 나타나도록(`typewriteText`, 깜빡이는 커서) 바꿨습니다. 대사 존재 자체를 모를 수 있다는 지적에는 첫 런 시작 시 대사 줄에 2.6초 스포트라이트 애니메이션을 넣고 인트로 문구에도 언급을 추가했습니다. 진짜 첫 런(`runs===0`)에만 뜨는 간단한 튜토리얼 오버레이도 추가했습니다 — 강화 픽과 같은 타임스탬프 밀어넣기 트릭을 재사용해 튜토리얼을 보는 시간이 60초 런타임에서 소비되지 않게 했고, 즉시 스킵할 수 있어 재방문 플레이어나 심사자의 진입 속도를 막지 않습니다.

검증: 첫 런 진입 시 튜토리얼이 뜨고 버튼 클릭으로 닫히는지(View Transition 완료 지연을 고려해 폴링으로 확인), 12초간 무작위로 5종 이상의 서로 다른 대사가 실제로 등장하는지 확인.

2.5 게임필(Juice) 보강

좀더 발전시켜보자, 정말 게임처럼 느껴지게. 너가 만족할 때까지, 비판적으로 체크해가며 작업해보자. 그 외에 재미있을 것 같은 것들을 넣어봐, 외부 리소스는 마음껏 써도 좋아.

두 차례에 걸쳐 Steve Swink의 『Game Feel』 원칙을 메커닉 변경 없이 순수 피드백 레이어로 적용했습니다. 1차: signal-field 중앙에 무결성과 연동되는 #core-orb (cyan→amber→rose, 정화·오클릭마다 반응, 파괴 시 shatter)를 추가하고, 오클릭 화면 흔들림·콤보 4/8/12 콜아웃·결과 화면 점수 카운트업/랭크 스탬프를 넣었습니다. 2차: 서버에이전트에게 현재 게임 구조를 제공하고 Vlambeer “The Art of Screenshake”, GDC “Juice It or Lose It”, 데스 리캡 UX 연구(Path of Exile, Minecraft)를 조사시켜 받은 8~12개 안 중 저비용·고효율 3개를 반영 — 정화 성공 시 시안색 글로우 펄스(오클릭 흔들림과 대칭되는 성공 쪽 피드백), 신호 만료 15% 이내 정화 시 CLUTCH 콜아웃, 결과 화면 “가장 아쉬웠던 순간” 리캡(시각·콤보·손실량).

마감 당일 추가로, 남겨 뒀던 리서치 백로그 중 저위험 3개를 마저 반영했습니다 — 결과 화면의 런 종료 빌드 요약(픽 순서대로 강화 나열), 코어 무결성이 마지막 한 칸일 때 화면 가장자리가 붉게 필싱하는 위험 비네트, 쿠키 스키마를 v3로 올려 추가한 연속 플레이 스트릭(하루 단위 갱신, 공백 시 1로 리셋). 데일리 시드 런·고스트 비교·콤보 계층 오디오는 새 저장 구조나 결정론적 시드가 필요해 마감 리스크가 커 계속 보류했습니다.

검증 중 발견한 버그 2건: (1) 결과 리캡 렌더 함수가 elements.resultRecap 엘리먼트 참조를 빠뜨려, 실수가 하나라도 있는 모든 런에서 결과 화면이 예외로 멈추는 버그 — 클린 런에서는 이 경로가 트리거되지 않아 겉으로는 통과하는 것처럼 보였습니다. 실제 실행 경로를 강제로 태우는 임시 디버그 훅(window.__debugFinishRun, 검증 후 즉시 제거)으로 발견·수정했습니다. (2) 클러치 히트 전용 초상화 에셋이 실제로는 존재하지 않아 매 클러치마다 404가 나던 버그 — 기존 sync-linked 초상화를 재사용하는 별칭 처리로 새 에셋 없이 해결했습니다. 두 버그 모두 배포 후 라이브 URL에서 재검증(콘솔 에러·404 0건).

2.6 메타 진행 — 기억 조각·쿠키·연속 플레이 스트릭

방문·완료 런·최고 점수·이전 결말·기억 조각(최대 6개)만 담는 1st-party 쿠키(state_less_memory_v1)로 재방문 진행도를 관리합니다. A랭크 이상으로 코어를 지키면 기억 조각과 MORI의 서사 기록(MORI.LOG)이 하나씩 열립니다. 오늘 추가한 연속 플레이 스트릭은 하루 단위 day-index로만 갱신되는 순수 함수(computeStreak)로 계산해, 같은 날 재접속은 유지되고 하루 공백은 0이 아닌 1로 리셋(오늘 접속 자체가 1일차)되도록 했습니다.

2.7 오디오 — 합성 SFX + Freesound CCO 배경 루프

정화·오클릭·경보·콤보·결과 등 대부분의 효과음은 Web Audio API 오실레이터로 그 자리에서 합성합니다. 배경 앰비언트 루프 1개만 참여자가 발급한 Freesound API 키로 CCO 라이선스 음원(“Industrial Machine Drone”, Freesound #854269)을 가져와 코어 무결성에 로우패스 필터가 연동되도록 붙였습니다(\$5.1). Token 인증만으로 미리듣기(HQ MP3) 다운로드가 가능해 OAuth2 로그인 없이 완료했고, API 키는 저장소에 포함하지 않고 로컬 환경 변수에만 저장했습니다.

2.8 화면 레이아웃 — 플레이 영역 비율 조정

메인 플레이 파트가 너무 화면에서 작은 걸 차지하는데.

배포 후 실사용 피드백으로, 플레이 영역(.signal-field)이 뷰포트 크기와 무관하게 고정 280px 높이여서 큰 화면에서는 슬롯이 작게 몰려 보인다는 지적을 받았습니다. 실제로 라이브 사이트를 여러 해상도(1920×1080, 1440×900, 390px 모바일)로 스크린샷 비교해 문제를 확인한 뒤, 높이를 clamp(280px, 46vh, 480px)로 바꿔 뷰포트에 비례하게 늘어나도록 하고 플레이·결과 화면 패널 폭도 1120px→1320px로 넓혔습니다. 슬롯 좌표가 이미 컨테이너 대비 퍼센트(--sx / --sy)로 정의돼 있어 위치 로직 변경 없이 자연스럽게 더 넓게 퍼졌습니다. 작은 화면에서는 clamp의 최소값이 기존 고정값과 같아 회귀가 없습니다.

3. 설계 판단 과정 — 여섯 번의 핵심 메커닉 전환

지금의 SIGNAL STRIKE에 도달하기까지 핵심 메커닉을 다섯 번 교체했습니다. 각 전환은 “재미없다”는 판단을 감이 아니라 이론·실제 플레이·소스 대조로 검증한 뒤에만 이뤄졌고, 이미 검증했다는 사실에 안주하지 않고 사용자의 재반박을 받아들여 다음 전환으로 이어갔습니다.

#	메커닉	전환 계기	검증 방법
1	SIGNAL LOCK (타이밍 미터 확정)	“쿠키 로그 해석 게임 같다”, “듀오링고 같다”	원인 진단: 6종 라운드도 실제 손동작은 항상 “읽고 아무 때나 클릭” 하나뿐

#	메커닉	전환 계기	검증 방법
2	SIGNAL TRIAGE (실시간 칩 분류)	MDA·Koster·Swink·Flow·Meaningful Play 5개 이론으로 진단 후 전면 재설계	서브에이전트가 텍스트 전용 하네스(가상 시계로 LLM 반응속도 문제 해결)로 실제 6웨이브 플레이
3	SESSION AUDIT (심문관형 교차 대조)	“속도만 오르고 판단 복잡도가 안 늘어난다”는 재지적 → Papers, Please 구조로 재진단	Puppeteer로 위조 승인 (-2)/오인(-1) 비대칭 페널티, 6웨이브 완주 실측
4	터미널 이동+도킹 레이어	“여전히 문서작업 같다” → 판정 로직 (순수 함수)은 재사용하고 도달 방식만 교체하는 절충안	Puppeteer로 이동·도킹·채점·놓침 5개 경로 직접 재현
5	SIGNAL STRIKE (현재 구조)	“이동도 결국 읽고 비교하는 문서작업” → 판정 자체를 색·기호 즉시 반응으로 교체	Puppeteer로 정화·오클릭·무결성 소진·랭크 판정 재검증

이 과정에서 AI가 스스로 내린 판단이 두 번 뒤집혔다는 점이 중요합니다 — SIGNAL TRIAGE는 5개 이론과 서브에이전트 플레이테스트로 “이전보다 재밌다”고 검증까지 마쳤지만 실제 플레이 앞에서 다시 기각됐고, 이동+도킹 레이어도 5개 경로를 전부 Puppeteer로 검증했지만 “검증되었다”와 “재밌다”는 다른 질문이라는 것을 다시 받아들여야 했습니다. 메타 진행 계층(무결성·기억 조각·ARCHIVE LENS·DEEP VERIFY)은 다섯 번의 전환 내내 “웨이브별 정답/오답/놓침 집계”라는 동일한 인터페이스만 소비했기 때문에 한 번도 다시 만들 필요가 없었습니다 — 판정 방식이 바뀌어도 그 결과를 담는 메타 구조는 초기 설계 그대로 재사용됐습니다.

4. 검증 방법론

- **실제 브라우저 조작:** 모든 기능은 puppeteer-core 로 로컬/헤드리스 Chrome을 직접 띄워 클릭·키보드 입력으로 재현해 검증합니다. “코드가 그렇게 짜여 있으니 동작할 것”이라는 추정으로 끝내지 않습니다.
- **서브에이전트 실패율:** SIGNAL TRIAGE 검증 시, 화면 녹화 없이 DOM 상태만 JSON으로 노출하는 하네스를 만들어 서브에이전트가 실제로 플레이하고 재미를 평가하게 했습니다. 이 과정에서 LLM의 느린 반응속도가 판정을 왜곡하는 문제를 performance.now() / requestAnimationFrame 을 가상 시계로 치환해 해결했습니다.
- **AI 주장의 재검증:** 서브에이전트가 보고한 버그나 개선안을 그대로 반영하지 않고, 소스 코드와 하나씩 대조해 사실 여부를 확인한 뒤에만 반영합니다.
- **디버그 흑으로 강제 경로 태우기:** RNG나 타이밍에 의존해 재현하기 어려운 경로(예: 실수가 있는 런의 결과 화면)는 window.__debugFinishRun 같은 임시 흑으로 강제 호출해 실제로 실행되는지 확인하고, 검증 직후 제거합니다. 이 방법으로 §2.5의 두 버그를 발견했습니다.
- **배포 후 라이브 재검증:** GitHub Pages 배포가 끝나면 로컬 빌드가 아니라 실제 라이브 URL에 대해 같은 시나리오를 다시 실행해 콘솔 에러·실패한 네트워크 요청이 0건인지 확인합니다.
- **전/후 상태 대조:** “동작한다”에 만족하지 않고, 상태 변경 전/후를 직접 스냅샷 비교합니다(§2.2의 보호막 표시 버그가 이 방법으로 발견됐습니다).

5. 외부 에셋과 오픈소스

항목	용도	버전	라이선스·출처
Vite	개발 서버와 프로덕션 번들	8.2.0	MIT, https://github.com/vitejs/vite

항목	용도	버전	라이선스·출처
OpenAI Codex	개발 보조(초기 ~SESSION AUDIT)	사용 환경 제공 버전	https://openai.com/codex/
Claude Code (Anthropic)	개발 보조(터미널 이동 +도킹 재설계 이후 전부)	사용 환경 제공 버전	https://www.anthropic.com/claude-code
puppeteer-core	실제 브라우저 조작 검증	package.json 명시 버전	Apache-2.0, https://github.com/puppeteer/puppeteer
Pandoc	제출용 PDF 생성 시 마크다운→HTML 변환 (게임 실행에는 미사용)	3.9.0	GPL-2.0-or-later, https://pandoc.org/
JetBrains Mono	일지·터미널 텍스트용 모노스페이스 폰트, <code>public/fonts/</code> 에 자체 호스팅	v24 (가변 폰트)	SIL Open Font License 1.1, https://github.com/JetBrains/JetBrainsMono
“Industrial Machine Drone (Seamless Loop)”	플레이 화면 배경 앰비언트 루프	Freesound #854269	CC0 1.0(퍼블릭 도메인), 제작자 kkenny101, https://freesound.org/s/854269/

MORI 전신 기준표와 상황별 흉상 이미지는 참여자가 기록된 프롬프트를 사용해 ChatGPT Image에서 생성·선택했습니다. UI 그래픽은 HTML/CSS로 직접 만들고, 대부분의 효과음은 Web Audio API 오실레이터로 합성합니다. 이 프로젝트는 “합성 오디오만 사용한다”를 원칙으로 못박아 두지 않았고, 라이선스가 명확하고 출처를 표에 남길 수 있는 외부 에셋이라면 필요에 따라 계속 추가할 수 있습니다.

5.1 배경 앰비언트 루프 추가(Freesound API)

Freesound API는 **Token 인증만으로 검색과 미리듣기(preview) 파일 다운로드가 가능**하고, 원본 파일 다운로드에만 OAuth2 로그인이 필요합니다. 게임에 쓰는 짧은 루프는 미리듣기 품질(HQ MP3, 128kbps)로도 충분해 OAuth2 없이 완료했습니다. 자격증명은 저장소에 포함하지 않고 참여자의 로컬 환경 변수에만 저장했습니다.

검색 기준은 CC0 라이선스 필터, `seamless loop` 태그, 게임 테마(신호·코어·SF 앰비언트)와의 어울림이었습니다. 최종 선택한 음원은 7.7초 완전 루프라 60초 런 내내 이어 붙여도 이음매가 들리지 않습니다. `AudioEngine` 에 `startDrone / setDroneTension / stopDrone` 을 추가해 반복 재생하고, 로우패스 필터 컷오프를 코어 무결성에, 게인을 남은 시간 비율에 연동했습니다 — 코어가 손상될수록 소리가 먹먹해지는 것은 코어 오브가 시각적으로 흐려지는 것과 같은 상태를 청각으로 반복하는 장치입니다.

6. 검증 명령

```

npm test           # 순수 게임 로직·쿠키 직렬화 테스트 22개
npm run build      # dist/ 정적 파일 생성
npm run check      # test + build
npm run docs:pdf   # 게임 소개 / AI 활용 기술 PDF 재생성

```

`npm run docs:pdf (scripts/build-docs-pdf.mjs)` 는 로컬 Pandoc으로 마크다운을 `pdf-style.css` 가 적용된 HTML로 변환한 뒤, 로컬 Chrome의 `page.pdf()` 로 A4 PDF를 생성합니다. 생성 후에는 매번 `pdftoppm` 으로 각 페이지를 이미지로 렌더해 잘림·빈 페이지·깨진 한글이 없는지 직접 확인합니다.

`scripts/record-game.mjs` (`npm run record`)는 SIGNAL STRIKE 이전 화면 구조(`#statement-board`, `.statement-chip` 등)를 자동으로 조작하도록 작성돼 있어 현재 화면 구조에서는 실행할 수 없습니다. 화면 구조가 여러 번 바뀌는 개발 단계에서는 매번 녹화 스크립트를 유지보수하는 비용이 임시 Puppeteer 스모크 테스트를 새로 짜는 비용보다 크다고 판단해 방치했습니다. 제출용 플레이 영상은 이 자동 녹화 대신 실제 사람이 화면을 직접 플레이하며 녹화합니다.